
rl-triton: High-Performance Triton GPU Kernels for Reinforcement Learning Credit Assignment

A PREPRINT

Lars Simon Zehnder
simon.zehnder@gmail.com

ABSTRACT

We present `rl-triton`, an open-source library of high-performance GPU kernels for reinforcement learning credit assignment, implemented in Triton. The core contribution is a unified associative scan framework that recasts seven distinct RL estimation algorithms –Generalized Advantage Estimation (GAE), V-Trace, Retrace(λ), TD(λ) returns, discounted returns, eligibility traces, and episodic prefix sums –as instances of a single first-order linear recurrence solved in $O(\log T)$ parallel steps. A single fused Triton kernel serves all algorithms by varying only the input tensors α and β , eliminating the $O(T)$ sequential dependency chain that plagues standard PyTorch loop implementations and `torch.compile` vectorized baselines. We provide correctness proofs via algebraic verification of the associative operator, precise episode-boundary semantics for both terminated and truncated episodes, and a benchmarking harness demonstrating consistent $\geq 1.5\times$ speedup over `torch.compile` at production batch sizes (128+ environments, 1024+ steps per rollout). The library is available at <https://github.com/simonsays1980/rl-triton>.

1 Introduction

Every iteration of a deep reinforcement learning training pipeline requires computing *credit assignment* quantities: advantages, value targets, and return estimates that tell the policy network which actions were good and by how much. These quantities include Generalized Advantage Estimation (GAE) [Schulman et al., 2016], V-Trace targets [Espeholt et al., 2018], Retrace(λ) Q-value targets [Munos et al., 2016], and several simpler estimators. While network forward and backward passes and environment rollouts typically dominate overall training time, credit assignment computations are on the critical path of every update step. They operate over tensors of shape `[num_envs, seq_len]`, where `num_envs` is the number of parallel environments (or workers) and `seq_len` is the rollout length. Typical configurations range from a handful of environments with short rollouts (e.g., 8 environments, 128 steps in standard PPO [Schulman et al., 2017]) to hundreds of environments with longer horizons in large-scale distributed training [Espeholt et al., 2018, Petrenko et al., 2020] and RLHF pipelines where rollout sequences span thousands of tokens [Ouyang et al., 2022] –making credit assignment an attractive target for kernel-level optimization across this wide range of scales.

All of these estimators share a common mathematical structure: they are **first-order linear backward recurrences** of the form

$$A_t = \alpha_t + \beta_t \cdot A_{t+1}, \quad A_T = 0 \text{ (or a bootstrap value)}, \quad (1)$$

where α_t and β_t are precomputed per-step scalars that depend on the specific algorithm. This sequential structure creates a strict data dependency chain: A_t cannot be computed until A_{t+1} is available. In a naive sequential implementation, T timesteps require T serial operations, regardless of how many parallel GPU cores are available.

Standard PyTorch implementations address this with vectorized operations that rewrite the recurrence as a series of element-wise multiplications and cumulative sums, or with `torch.compile` which automates similar transformations. The vectorized parallel scan requires $O(\log T)$ sequential kernel launches, each issuing a full HBM read and write of the entire tensor. Individual passes do use memory bandwidth efficiently, but the $O(\log T)$ round-trips accumulate: for $T=1024$ that is ten complete HBM passes, versus two (one load, one store) for a fused kernel [Dao et al., 2022].

We propose a different approach: a **fused Triton kernel** that computes each algorithm’s credit assignment quantities in a single pass over HBM, performing all intermediate arithmetic entirely within GPU registers and shared memory (SRAM). The kernel exploits a classical result from parallel computing – the **parallel associative scan** [Blelloch, 1990] – to restructure the $O(T)$ sequential recurrence into an $O(\log T)$ parallel tree reduction. Each environment row is assigned to one SM; within that SM, all reduction passes operate entirely in SRAM and registers, with no HBM access until the final output write. Across the batch, all environment rows execute on separate SMs simultaneously.

The key insight is that all seven credit assignment primitives implemented in `rl-triton` reduce to the same first-order recurrence (Equation 1) under appropriate substitutions for α_t and β_t . This means a single Triton scan kernel, parameterized only by its inputs, correctly implements GAE, V-Trace, Retrace(λ), TD(λ) returns, discounted returns, eligibility traces, and episodic prefix sums. The algorithms differ only in the PyTorch preprocessing layer that constructs the α and β tensors from raw rollout data.

Contributions.

1. A unified algebraic framework showing that seven standard RL credit assignment algorithms are instances of a single first-order linear recurrence.
2. A fused Triton kernel implementing the parallel associative scan with full episode-boundary semantics, including both terminated and truncated episodes.
3. Precise treatment of the bootstrap value at rollout window boundaries, which appears in two mathematically distinct roles – as an additive term in α_{T-1} (the TD residual) and as the scan boundary carry A_T – and must be handled consistently.
4. A benchmarking harness showing $\geq 1.5\times$ speedup over `torch.compile` on standard PPO-scale workloads.
5. An open-source Python package with correctness tests, performance safeguard tests, and complete documentation.

2 Background

2.1 The Sequential Dependency Problem

Consider the GAE recurrence [Schulman et al., 2016]:

$$A_t = \delta_t + \gamma \lambda \cdot A_{t+1},$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the one-step TD error. Computing A_0 requires A_1 , which requires A_2 , all the way to A_{T-1} . This is a strict sequential chain of length T .

On GPU hardware, sequential chains of length T are expensive not because of arithmetic cost but because of **memory traffic**: a naive loop must read δ_t and $\gamma \lambda$ from HBM, perform the addition and multiplication, and write A_t back to HBM – one full round-trip per timestep. For $T=1024$, `num_envs=128`, this means 1024 serial HBM round-trips even though 128 environment rows could in principle be processed in parallel.

`torch.compile` can vectorize across the environment dimension but cannot eliminate the sequential dependency along the time dimension. It still requires $O(T)$ serial steps to compute each environment’s advantages, even though the arithmetic at each step is cheap.

2.2 Parallel Associative Scans

The parallel associative scan is a foundational algorithm in parallel computing [Blelloch, 1990, Ladner and Fischer, 1980]. Given an array of elements $[x_0, x_1, \dots, x_{T-1}]$ and an associative binary operator $\oplus$, the scan computes all prefix reductions $[x_0, x_0 \oplus x_1, \dots, x_0 \oplus \dots \oplus x_{T-1}]$ in $O(\log T)$ parallel steps on T processors, using a binary tree reduction.

The critical observation for RL credit assignment is that the first-order linear recurrence $A_t = \alpha_t + \beta_t \cdot A_{t+1}$ can be recast as computing the prefix scan of tuples (α_t, β_t) under the associative operator:

$$(\alpha_B, \beta_B) \oplus (\alpha_A, \beta_A) = (\alpha_B + \beta_B \alpha_A, \beta_A \beta_B). \quad (2)$$

Proposition 1 (Associativity of $\oplus$). *The operator defined by Equation 2 is associative.*

Proof. For any three tuples $(\alpha_C, \beta_C), (\alpha_B, \beta_B), (\alpha_A, \beta_A)$:

$$\begin{aligned} [(\alpha_C, \beta_C) \oplus (\alpha_B, \beta_B)] \oplus (\alpha_A, \beta_A) &= (\alpha_C + \beta_C \alpha_B, \beta_B \beta_C) \oplus (\alpha_A, \beta_A) \\ &= (\alpha_C + \beta_C \alpha_B + \beta_B \beta_C \alpha_A, \beta_A \beta_B \beta_C). \\ (\alpha_C, \beta_C) \oplus [(\alpha_B, \beta_B) \oplus (\alpha_A, \beta_A)] &= (\alpha_C, \beta_C) \oplus (\alpha_B + \beta_B \alpha_A, \beta_A \beta_B) \\ &= (\alpha_C + \beta_C (\alpha_B + \beta_B \alpha_A), \beta_A \beta_B \beta_C) \\ &= (\alpha_C + \beta_C \alpha_B + \beta_C \beta_B \alpha_A, \beta_A \beta_B \beta_C). \end{aligned}$$

Both expressions are equal. $\square$

Because the grouping in the scan does not affect the result, the tree reduction is mathematically sound. In the kernel, the sequence is loaded in reverse chronological order (index 1 = last timestep $t=T-1$, index T = first timestep $t=0$), so the scan sweeps from late to early time. After the scan, the tuple at reversed index k holds the correct advantage value for chronological timestep $t = T - k$.

2.3 Triton and the GPU Memory Hierarchy

Triton [Tillet et al., 2019] is a domain-specific language and compiler for writing GPU kernels in Python-like syntax. It abstracts over warp-level programming while exposing the key performance-relevant features of the GPU memory hierarchy:

- **HBM (High Bandwidth Memory):** Main GPU memory, $\sim 1\text{--}3$ TB/s bandwidth, high latency (~ 400 cycles). Accessible by all SMs.
- **SRAM (Shared Memory):** Per-SM scratchpad, $\sim 5\text{--}10\times$ higher bandwidth than HBM, very low latency. Capacity ~ 100 KB per SM.
- **Registers:** Per-thread storage, zero-latency access. Arithmetic occurs here.

Credit assignment kernels are memory-bandwidth bound rather than compute bound [Williams et al., 2009]: the arithmetic intensity (FLOPs per byte of HBM traffic) is low, so reducing HBM accesses directly reduces runtime. The performance advantage of a fused kernel therefore comes from **minimizing HBM round-trips** [Dao et al., 2022]. A fused kernel loads all inputs once, performs all computation in registers and SRAM, and stores the results once. The associative scan operates entirely within SRAM: threads exchange intermediate tuples via warp shuffles and SRAM writes during the $O(\log T)$ reduction passes, with no return to HBM until the final output store.

Triton’s `tl.associative_scan` primitive maps directly to this pattern, allowing the kernel to express the tree reduction at a high level while the Triton compiler generates efficient hardware-specific reduction instructions for both NVIDIA (warp shuffles) and AMD (wavefront operations) GPUs.

3 The Unified Linear Recurrence Framework

3.1 The Shared Kernel

All credit assignment algorithms in `rl-triton` are implemented by the same low-level Triton kernel, which solves:

$$A_t = \alpha_t + \beta_t \cdot A_{t+1}, \quad A_T = \text{bootstrap}.$$

The backward scan kernel loads (α_t, β_t) in reverse time order, applies `tl.associative_scan` with the combine function (Equation 2), adds the bootstrap boundary condition, and writes the output in original time order. The forward scan kernel for eligibility traces uses the same combine function but loads in natural time order.

All Python-level API functions dispatch to a fused kernel for the common case (`seq_len` $\leq 131\,072$), which receives the raw rollout tensors (rewards, values, done flags, importance ratios, etc.) and derives α_t and β_t in-register – no PyTorch step materializes them as intermediate tensors. For longer sequences the functions fall back to a chunked PyTorch path that does precompute α_t and β_t as full HBM tensors before invoking the scan. Table 1 summarizes the per-algorithm mapping of rollout quantities to α_t and β_t .

3.2 Episode Boundary Semantics

A central correctness requirement is that credit assignment quantities never propagate across episode boundaries within a rollout buffer. `rl-triton` distinguishes two types of boundaries with distinct handling.

Table 1: Algorithm-specific inputs to the universal backward recurrence $A_t = \alpha_t + \beta_t \cdot A_{t+1}$. $d_t^{\text{term}} \in \{0, 1\}$ indicates true episode termination; $d_t^{\text{trunc}} \in \{0, 1\}$ indicates truncation (time limit); $d_t = d_t^{\text{term}} \vee d_t^{\text{trunc}}$. $\Delta_t = v_t - V(s_t)$ is the V-Trace value delta (see Section 3.3.2). Using d_t (not d_t^{term}) in β_t is intentional. Both truncation and the window boundary represent an ongoing episode whose continuation is not in the buffer, requiring a bootstrap value $b = V(s_{t+1}^{\text{true}})$ [Pardo et al., 2018]. The two cases differ only mechanically: at a truncated *interior* step, the next buffer entry belongs to a different episode, so $\beta_t = 0$ actively severs the scan carry to prevent that data from propagating, and b_t is absorbed into α_t directly. At the rollout *window boundary* there is nothing after $T-1$ in the buffer, so no severing is needed; $\beta_{T-1} > 0$ and the bootstrap $b = V(s_T)$ is supplied as the external scan carry A_T , which propagates back through $\prod_k \beta_k$ in the post-scan step (see Section 3.2).

Algorithm	α_t	β_t	Bootstrap
GAE	$r_t + \gamma(1 - d_t^{\text{term}})V(s_{t+1}) - V(s_t)$	$\gamma\lambda(1 - d_t)$	A_T
V-Trace	$\rho_t(r_t + \gamma(1 - d_t^{\text{term}})V(s_{t+1}) - V(s_t))$	$\gamma c_t(1 - d_t)$	Δ_T
Retrace(λ)	$r_t + \gamma(1 - d_t^{\text{term}})\mathbb{E}_\pi[Q(s_{t+1}, \cdot)] - Q(s_t, a_t)$	$\gamma c_{t+1}(1 - d_t)$	0
TD(λ)	$r_t + \gamma(1 - \lambda)(1 - d_t^{\text{term}})V(s_{t+1})$	$\gamma\lambda(1 - d_t)$	G_T^λ
Disc. Returns	r_t	$\gamma(1 - d_t)$	G_T
Elig. Traces (fwd)	$\nabla_{\mathbf{w}} \hat{V}(s_t, \mathbf{w}_t)$	$\gamma\lambda(1 - d_t)$	$\mathbf{z}_{-1} = \mathbf{0}$
Prefix Sum (fwd)	x_t	$1 - d_t$	0

Terminated steps ($d_t^{\text{term}} = 1$). The episode ended naturally. The value of the terminal successor state $V(s_{t+1})$ is conceptually zero, so it is zeroed out of the TD error via the factor $(1 - d_t^{\text{term}})$. The decay β_t is zeroed via $(1 - d_t)$, severing the scan carry.

Truncated steps ($d_t^{\text{trunc}} = 1$). The episode was cut short by a time limit but continues in principle. The successor state $V(s_{t+1})$ stored in the values buffer belongs to a different episode and must not be used. Instead, the caller supplies the correct continuation value via `bootstrap_values[env, t]`, which is substituted for `values[t+1]` in the TD error computation – so the bootstrap enters α_t with the γ discount already applied. The decay β_t is zeroed via $(1 - d_t)$, which severs the scan carry from the next episode’s data in the buffer; it does not affect the bootstrap already encoded in α_t .

Window boundary (final step at $t = T-1$, carry A_T). The final step of the rollout buffer may end mid-episode. The successor state $V(s_T)$ lies one step past the buffer. This value appears in two places simultaneously: (1) in α_{T-1} as the TD residual bootstrap, and (2) as the scan carry A_T . After the reduction, the carry propagates backward to every position $t \in \{0, \dots, T-1\}$ via the accumulated decay product:

$$A_t = (\text{local scan result at } t) + \left(\prod_{k=t}^{T-1} \beta_k \right) \cdot A_T.$$

Since both uses read the same scalar, a single entry `bootstrap_values[env, T-1]` satisfies both roles. The kernel reads this value once and uses it in both places.

For the common case where episodes terminate at the rollout boundary ($d_{T-1}^{\text{term}} = 1$), no bootstrap is needed and the kernel uses $A_T = 0$ without allocating any additional tensor.

3.3 Algorithm-Specific Details

The following subsections specify α_t and β_t for each algorithm. Episode-boundary semantics – how terminated steps, truncated steps, and the window boundary are handled – are common to all algorithms and described in Section 3.2; they are not repeated here.

3.3.1 Generalized Advantage Estimation (GAE)

GAE [Schulman et al., 2016] computes the advantage as the exponentially-weighted mixture of n -step TD residuals:

$$A_t^{\text{GAE}} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}.$$

This telescopes to the backward recurrence $A_t = \delta_t + \gamma\lambda \cdot A_{t+1}$. With episode-boundary masking:

$$\delta_t = r_t + \gamma(1 - d_t^{\text{term}})V(s_{t+1}) - V(s_t), \quad \beta_t = \gamma\lambda(1 - d_t).$$

The $\lambda = 0$ case reduces GAE to a pure one-step TD advantage (low variance, high bias); $\lambda = 1$ yields the full Monte Carlo return minus the baseline (low bias, high variance).

3.3.2 V-Trace

V-Trace [Espeholt et al., 2018] corrects for policy lag in distributed training by weighting each TD error with a clipped importance sampling (IS) ratio. Let π be the current learner policy and μ the behaviour policy that collected the data. Define the per-step IS ratio $\rho_t^{\text{raw}} = \pi(a_t|s_t)/\mu(a_t|s_t)$, then clip it at two separate thresholds:

$$\rho_t = \min(\bar{\rho}, \rho_t^{\text{raw}}), \quad c_t = \min(\bar{c}, \rho_t^{\text{raw}}).$$

$\bar{\rho}$ controls the bias-variance trade-off of the value target: a smaller $\bar{\rho}$ limits how far the target can deviate from the on-policy value, reducing variance at the cost of some bias. $\bar{c}$ controls the multi-step trace length: it multiplies along the return trajectory in $\beta_t = \gamma c_t(1 - d_t)$, so a smaller $\bar{c}$ shortens the effective lookahead, trading variance for bias. Typical defaults are $\bar{\rho} = 1$ and $\bar{c} = 1$ [Espeholt et al., 2018]. The V-Trace target delta satisfies $\Delta_t = \delta_t^V + \gamma c_t \cdot \Delta_{t+1}$, mapping to:

$$\alpha_t = \rho_t(r_t + \gamma(1 - d_t^{\text{term}})V(s_{t+1}) - V(s_t)), \quad \beta_t = \gamma c_t(1 - d_t).$$

After the scan, targets are formed as $v_t = \Delta_t + V(s_t)$ and advantages as $A_t = \rho_t(r_t + \gamma v_{t+1} - V(s_t))$, all computed within the same fused kernel.

3.3.3 Retrace(λ)

Retrace(λ) [Munos et al., 2016] extends V-Trace to Q-value estimation with an *index shift*: the decay coefficient applying to Δ_{t+1} is c_{t+1} (next-step importance weight) rather than c_t :

$$\Delta_t = \delta_t + \gamma c_{t+1} \cdot \Delta_{t+1}.$$

This shift reflects that Retrace evaluates a state-action pair (s_t, a_t) : the action a_t is fixed as the subject of evaluation, so off-policy correction begins only from the next action a_{t+1} onward. The kernel obtains c_{t+1} without pre-shifting the probability array: because each lane loads at reversed index $\text{rev} = T-1-\text{offs}$, the real array index of $t+1$ is simply $\text{rev}+1$, so a second pointer-offset load in the same kernel gives $\pi(a_{t+1}|s_{t+1})$ and $\mu(a_{t+1}|s_{t+1})$ directly in registers. At the boundary lane ($\text{offs}=0, t=T-1$), c_T is undefined and is masked to zero.

The boundary carry is fixed at zero ($\Delta_T = 0$): the expected Q-values at s_T are already encoded in α_{T-1} , and $\beta_{T-1} = \gamma c_T(1 - d_{T-1})$ contains c_T , which requires the behaviour policy probability of the action taken at s_T – an action that was never sampled, so $c_T = 0$ by convention. Interior values Δ_t are non-zero in general and computed by the scan.

3.3.4 TD(λ) Returns and Discounted Returns

TD(λ) returns [Sutton and Barto, 2018] interpolate between one-step TD ($\lambda = 0$) and Monte Carlo ($\lambda = 1$):

$$G_t^\lambda = r_t + \gamma[(1 - \lambda)V(s_{t+1}) + \lambda G_{t+1}^\lambda].$$

Expanding and grouping into $\alpha_t + \beta_t \cdot G_{t+1}^\lambda$ form:

$$\alpha_t = r_t + \gamma(1 - \lambda)(1 - d_t^{\text{term}})V(s_{t+1}), \quad \beta_t = \gamma\lambda(1 - d_t).$$

Discounted returns [Sutton and Barto, 2018] are the special case $\lambda = 1$ with no value function: $\alpha_t = r_t, \beta_t = \gamma(1 - d_t)$. Two boundary cases require separate treatment. At *interior truncated steps* ($d_t^{\text{trunc}} = 1, t < T-1$), $\beta_t = 0$ severs the carry, so the continuation value must enter α_t directly: $\alpha_t = r_t + \gamma V(s_{t+1}^{\text{true}})$. At the *window boundary* ($t = T-1$, continuing episode), $\beta_{T-1} = \gamma > 0$ and the bootstrap $V(s_T)$ is passed as the scan carry $G_T = V(s_T)$, giving $G_{T-1} = r_{T-1} + \gamma V(s_T)$ – the correct one-step bootstrapped return at the rollout edge.

3.3.5 Eligibility Traces

The eligibility trace is the only **forward** recurrence in the library:

$$\mathbf{z}_t = \gamma\lambda(1 - d_t)\mathbf{z}_{t-1} + \nabla_{\mathbf{w}}\hat{V}(s_t, \mathbf{w}_t).$$

This maps to $\alpha_t = \nabla_{\mathbf{w}}\hat{V}(s_t, \mathbf{w}_t), \beta_t = \gamma\lambda(1 - d_t)$. Unlike the backward algorithms, the sequence is loaded in *chronological* order ($t = 0 \rightarrow T-1$, past to present) so each position accumulates a weighted sum of past gradients rather than future TD errors. The same combine function applies; only the load order differs. For linear function approximation, $\nabla_{\mathbf{w}}\hat{V}(s_t, \mathbf{w}_t) = \mathbf{x}_t$ (the feature vector).

3.3.6 Episodic Prefix Sum

The episodic prefix sum (segmented scan) computes a running total that resets at episode boundaries:

$$C_t = x_t + (1 - d_t) C_{t-1},$$

mapping to $\alpha_t = x_t, \beta_t = 1 - d_t$. This primitive underlies episode-level return logging, per-step timestep indices for time-aware RL [Pardo et al., 2018], cumulative distribution functions for prioritized replay sampling [Schaul et al., 2016], and sequence-packing position IDs in LLM pre-training.

4 Implementation

4.1 Kernel Architecture

Each environment row in the batch is processed by a separate Triton program instance (one program per row, indexed by `tl.program_id(0)`), which the GPU scheduler maps to one Streaming Multiprocessor (SM). Within a program, `BLOCK_SIZE` threads execute concurrently on that SM – one thread per timestep – operating entirely in SRAM and registers [Kirk and Hwu, 2016].

The generic backward scan kernel takes pre-computed α and β tensors as input. The fused kernels for GAE, V-Trace, and Retrace instead load raw rollout tensors (rewards, values, done flags) and compute α and β in-register, eliminating intermediate HBM writes – see Section 4.2. For both paths the kernel procedure is:

1. **Load** (coalesced HBM $\rightarrow$ registers): Load input tensors for all T timesteps in reverse time order using `tl.arange` offset arithmetic [Kirk and Hwu, 2016].
2. **Compute** α, β (registers): Fused kernels derive α_t and β_t from raw inputs here; the generic kernel uses the loaded values directly.
3. **Scan** (registers/SRAM): Apply `tl.associative_scan(( $\alpha, \beta$ ), axis=0, combine_fn=combine)`. `BLOCK_SIZE` is a compile-time `constexpr` power of 2, so the reduction executes exactly $\log_2(\text{BLOCK_SIZE})$ passes – a fixed loop fully unrolled by the compiler. Out-of-range lanes are masked automatically inside the kernel via `tl.load/tl.store` masks; no caller preprocessing is required.
4. **Bootstrap** (registers): Add the carry A_T to each position: `out = scan_result + decay_product * bootstrap`.
5. **Store** (registers $\rightarrow$ HBM): Write all T results in original time order using the same reversed offsets.

4.2 The Fused Kernel Pattern

The fused kernel for GAE, V-Trace, and Retrace computes α and β from raw rollout tensors in-register, inside the same Triton program, so those intermediate values are never written to HBM. Table 2 compares the HBM access count for GAE with bootstrap values against an unfused pipeline that materializes δ and β as full tensors between steps.

Table 2: HBM accesses for GAE (with bootstrap values, no truncations). r : rewards; v : values; d : terminateds; bs : bootstrap values; δ : TD residuals; β : decay scalars; A : advantages.

Step	Reads	Writes
<i>Fused kernel (single Triton program)</i>		
Load r, v_t, v_{t+1} (same tensor, two HBM reads at offsets <code>rev</code> and <code>rev+1</code>), d, bs ; compute in-register; write A	5	1
Total		6
<i>Unfused pipeline (three separate PyTorch ops)</i>		
Compute $\delta = r + \gamma v_{t+1}(1-d) - v_t$	5	1
Compute $\beta = \gamma\lambda(1-d)$	1	1
Scan: $(\delta, \beta) \rightarrow A$	2	1
Total		11

The fused kernel saves 5 HBM accesses by eliminating the intermediate write and subsequent re-read of δ and β .

The fused kernel uses compile-time `constexpr` flags to specialize for the common case (no truncations, no bootstrap value):

- `HAS_TRUNCATIONS=False`: skips the truncateds array and the 2D `bootstrap_values[num_envs, seq_len]` tensor (used for per-step truncation bootstraps), saving two HBM reads. The window-boundary bootstrap is still supported via a separate 1D scalar argument `bootstrap_ptr[num_envs]`, controlled by `HAS_BOOTSTRAP`.
- `HAS_BOOTSTRAP=False`: substitutes the literal 0.0 for the window-boundary bootstrap scalar, skipping even the 1D read (the common case when the rollout ends with a terminal state). Measured at $\sim 33\%$ of total kernel time at 128×1024 .

4.3 Chunked Fallback for Long Sequences

The flat kernel is limited to `seq_len` $\leq$ 131072 (the maximum `BLOCK_SIZE` supported by Triton’s `tl.associative_scan`). For longer sequences, backward algorithms (GAE, V-Trace, Retrace, TD(λ), discounted returns) fall back to a chunked PyTorch path that processes the sequence in blocks and propagates the carry between blocks. The eligibility trace forward scan does not have a chunked fallback: a left-to-right chunked forward scan is structurally feasible but was not implemented because sequences longer than 131072 timesteps are very uncommon in on-policy RL, making the added implementation complexity hard to justify.

The chunked path is unfused: it materializes α and β as full tensors before invoking the scan. At `seq_len` $>$ 131072, throughput is dominated by global memory bandwidth rather than kernel launch overhead, so the fusion benefit is smaller and the additional implementation complexity of a fused chunked kernel is not justified for typical RL workloads.

4.4 Numerical Precision

All kernels require `float32` inputs and raise a `TypeError` on other dtypes. The scan accumulates T additions over potentially thousands of timesteps; `bfloat16`’s 7 mantissa bits cause meaningful numerical drift at large T compared to `float32`’s 23 mantissa bits. In mixed-precision training pipelines (`torch.autocast`), the recommended pattern is to cast advantage inputs to `float32` before the scan and allow the policy forward pass to run in `bfloat16`.

5 Correctness Analysis

5.1 Algebraic Verification

The correctness of the associative scan for the backward recurrence is verified by tracing the 4-step tree reduction and confirming the accumulated tuple matches the sequential computation. For a 4-step sequence, the array is loaded in reverse chronological order: thread i initially holds $(\alpha^{(i)}, \beta^{(i)})$, where superscripts denote reversed array positions (1 = last timestep $t=3$, 4 = first timestep $t=0$). After the scan, thread i holds $T_{1..i}$, the prefix result covering reversed positions 1 through i .

Reduction pass 1 (stride 1). Thread 1 has no left neighbour and keeps its initial tuple. Each subsequent thread i combines with thread $i-1$ and stores the result as $T_{1..i}$:

$$\begin{aligned} T_{1..1} &= (\alpha^{(1)}, \beta^{(1)}), \\ T_{1..2} &= T_{1..2}^{\text{init}} \oplus T_{1..1} = (\alpha^{(2)} + \beta^{(2)}\alpha^{(1)}, \beta^{(1)}\beta^{(2)}), \\ T_{1..3} &= T_{1..3}^{\text{init}} \oplus T_{1..2} = (\alpha^{(3)} + \beta^{(3)}\alpha^{(2)}, \beta^{(2)}\beta^{(3)}), \\ T_{1..4} &= T_{1..4}^{\text{init}} \oplus T_{1..3} = (\alpha^{(4)} + \beta^{(4)}\alpha^{(3)}, \beta^{(3)}\beta^{(4)}). \end{aligned}$$

Reduction pass 2 (stride 2). Thread 4 combines with Thread 2’s accumulated tuple $T_{1..2}$:

$$T_{1..4} \leftarrow T_{1..4} \oplus T_{1..2} = (\alpha^{(4)} + \beta^{(4)}\alpha^{(3)} + \beta^{(4)}\beta^{(3)}\alpha^{(2)} + \beta^{(4)}\beta^{(3)}\beta^{(2)}\alpha^{(1)}, \beta^{(1)}\beta^{(2)}\beta^{(3)}\beta^{(4)}).$$

With `BLOCK_SIZE` = 4, exactly $\log_2 4 = 2$ passes are needed and the loop terminates. In general the loop executes exactly $\log_2(\text{BLOCK_SIZE})$ fixed passes – no dynamic stopping criterion.

The α component of $T_{1..4}$ equals the sequential expansion $A^{(4)} = \alpha^{(4)} + \beta^{(4)}(\alpha^{(3)} + \beta^{(3)}(\alpha^{(2)} + \beta^{(2)}\alpha^{(1)}))$, confirming correctness. Reversed position 4 corresponds to chronological $t=0$, so $T_{1..4}$ holds the advantage for the *first* timestep, which depends on all four TD residuals. Each thread accumulates its full prefix by snapping pre-computed sub-results together logarithmically, without waiting on any other thread sequentially.

5.2 Episode Boundary Correctness

When $d_t = 1$ at a boundary step (zeroing $\beta_t = 0$), the associative operator severs the carry. In the operator $(\alpha_B, \beta_B) \oplus (\alpha_A, \beta_A)$, B carries the contribution of chronologically later timesteps and A the earlier ones. With $\beta_B = 0$:

$$(\alpha_B, 0) \oplus (\alpha_A, \beta_A) = (\alpha_B + 0 \cdot \alpha_A, \beta_A \cdot 0) = (\alpha_B, 0).$$

The contribution of all earlier timesteps (α_A) is multiplied out, and the β product is zeroed, so the boundary cannot propagate further into the past. The scan result at any timestep t is therefore unaffected by data from future episodes separated by a done flag.

6 Benchmarking Methodology

6.1 Measurement Protocol

All benchmarks use CUDA event timing with the following protocol:

1. **Warmup** (20 iterations, untimed): absorbs Triton JIT compilation, autotuning, and first-touch allocation.
2. **Timed iterations** (50 per trial, 5 trials): each iteration is individually timed with CUDA event synchronization before start and after stop.
3. **Robust estimator**: each trial reports its median over 50 iterations; the final result is the minimum of 5 trial medians.

The min-of-medians estimator follows established parallel benchmarking practice [Hoeffler and Belli, 2015]. GPU hardware can exhibit occasional multi-millisecond interference bursts – such as frequency-scaling transitions or OS scheduling stalls – that inflate an entire trial’s measurements. Because a fixed-overhead stall affects short kernels proportionally more than long ones, this can produce non-monotonic results where a small batch (e.g., 1 env, 100 steps, nominally 0.3 ms) measures *slower* than a large batch (e.g., 128 envs, 1024 steps, nominally 5 ms) if the stall coincides with the small-batch trial. Taking the minimum of 5 independently warmed trial medians is a sound estimator of *steady-state cost* – the kernel’s runtime under stable GPU frequency and memory state, free of cold-start artifacts – and eliminates these non-monotonic outliers.

6.2 Baseline

The comparison baseline is `torch.compile` applied to a vectorized PyTorch implementation of the parallel suffix scan – the same $O(\log T)$ doubling algorithm implemented as a series of vectorized shift-and-add operations over `[num_envs, T]` tensors. This baseline represents the best practically achievable performance from within PyTorch’s eager + compile framework, without custom Triton kernels.

6.3 Performance Target

The library ships with a performance safeguard test (`pytest -m perf`) that asserts a minimum $1.5\times$ speedup of the `rl-triton` kernel over `torch.compile` at a representative configuration (128 environments, 1024 steps) for each algorithm. This test runs in CI to detect regressions.

7 Related Work

RL frameworks. CleanRL [Huang et al., 2022], RLlib [Liang et al., 2018], and Sample Factory [Petrenko et al., 2020] implement GAE and return computation in PyTorch. These implementations use sequential loops or vectorized cumulative product patterns that do not exploit the parallel scan structure. `rl-triton` provides drop-in replacements for these computations.

Parallel scans in ML. The use of associative scans for recurrent sequence modeling has been studied extensively in the context of state space models. S4 [Gu et al., 2022] and Mamba [Gu and Dao, 2023] exploit parallel scans for linear recurrent layers. Smith et al. [2023] introduce Linear Recurrent Units (LRUs) – a simplified linear recurrent architecture – and use parallel scans to accelerate their forward pass; LRUs have been adopted as policy-network backbones in RL settings requiring memory over observations. `rl-triton` applies the same parallel scan machinery to *credit assignment* rather than sequence modeling: a distinct use case (post-rollout, not forward-pass) that is structurally identical.

Triton kernels for ML. FlashAttention [Dao et al., 2022, Dao, 2023] demonstrates that fused Triton kernels can achieve order-of-magnitude speedups over standard attention implementations by eliminating HBM round-trips. rl-triton applies the same fusion principle to RL credit assignment.

Off-policy correction. V-Trace [Espeholt et al., 2018] and Retrace(λ) [Munos et al., 2016] are the off-policy correction algorithms implemented in rl-triton. ACER [Wang et al., 2017] (Actor-Critic with Experience Replay) uses a related truncated importance-sampling correction with a trust-region stabilizer but is not currently implemented. All three algorithms share the recursive importance-weighted return structure that makes the parallel scan applicable.

8 Discussion and Future Work

8.1 Applicability Beyond Standard RL

Several algorithms outside standard RL training share the same first-order recurrence structure handled by rl-triton:

- **RLHF pipelines:** GAE is used as the advantage estimator in PPO-based RLHF [Ouyang et al., 2022, e.g., InstructGPT], where rollout sequences can be hundreds to thousands of tokens long.
- **GRPO:** Group Relative Policy Optimization [Shao et al., 2024] uses discounted reward-to-go computed by `compute_discounted_returns`.
- **Sequence packing:** LLM pre-training pipelines pack multiple variable-length documents into a single flat GPU sequence to avoid padding waste [Krell et al., 2022]. Each token requires a *local* position ID (tokens elapsed since the last document boundary) for RoPE [Su et al., 2024]. Because `compute_episodic_prefix_sum` operates on scalar-valued inputs of shape `[num_envs, seq_len]`, it computes these indices directly: passing $x_t=1$ (float32 ones) with document boundaries as `done`s yields the per-token within-document step count, which the caller casts to `int64` for use as RoPE position indices. The same need arises in vision-language models (VLMs) with interleaved image-text sequences, where image-patch and text tokens carry separate local position indices.
- **Prioritized experience replay:** The same prefix sum kernel computes the CDF over a flat priority array for GPU-side parallel sampling via binary search [Schaul et al., 2016].

8.2 Limitations

Sequence length limit. The flat fused kernel is limited to `seq_len` ≤ 131072 by Triton’s internal block size constraints. Longer sequences fall back to a two-pass chunked path that is unfused and incurs additional HBM traffic.

Float32 only. All kernels require `float32` inputs. `bfloat16` support requires careful analysis of precision tradeoffs across algorithms and sequence lengths; it is deferred to a future release.

Forward scan (eligibility traces). The eligibility trace forward scan has no chunked fallback and is limited to `seq_len` ≤ 131072 .

Scalar β only. The current framework assumes β_t is a scalar per timestep. Algorithms with vector-valued decay – such as per-parameter eligibility traces [van Seijen et al., 2016, Sutton and Barto, 2018], where each network parameter carries its own trace – require element-wise $\beta_t \in \mathbb{R}^d$ and a different kernel structure where each dimension carries its own decay product. Whether eligibility traces improve deep RL (where per-parameter traces interact with the nonlinearity of the network) remains an active research question [Framling, 2007]; extending rl-triton to support vector-valued β_t is a natural direction once that literature matures.

8.3 Planned Extensions

v0.2 plans multi-block hierarchical 2D grid scans for low-environment / high-sequence configurations, fused advantage normalization, `bfloat16` support, and CUDA graph support for small-batch workloads. v0.3 plans differentiable loss kernels (PPO clipped surrogate, GRPO, KL divergence estimators) with hand-written backward passes.

9 Conclusion

rl-triton demonstrates that seven standard RL credit assignment algorithms –GAE, V-Trace, Retrace(λ), TD(λ) returns, discounted returns, eligibility traces, and episodic prefix sums –share a common first-order linear recurrence structure that enables a single parallel associative scan Triton kernel to serve all of them. The unified framework eliminates $O(T)$ sequential HBM round-trips, replacing them with a single coalesced load, an $O(\log T)$ in-SRAM tree reduction, and a single coalesced store. The result is a $\geq 1.5\times$ speedup over `torch.compile` at production batch sizes, with full correctness across terminated episodes, truncated episodes, and rollout window boundaries.

References

- Guy E. Blelloch. Prefix sums and their applications. Technical Report CMU-CS-90-190, School of Computer Science, Carnegie Mellon University, 1990.
- Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14135.
- Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning (ICML)*, 2018. arXiv:1802.01561.
- Kary Framling. Replacing eligibility traces for action-value learning with function approximation. In *Proceedings of the 15th European Symposium on Artificial Neural Networks (ESANN)*, pages 345–350, 2007.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2111.00396.
- Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC’15)*, 2015.
- Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G. M. Araújo. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.
- David B. Kirk and Wen-mei W. Hwu. *Programming Massively Parallel Processors: A Hands-On Approach*. Morgan Kaufmann, 3rd edition, 2016.
- Mario Michael Krell, Matej Kosec, Sergio P. Perez, and Andrew Fitzgibbon. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting model quality. In *NeurIPS 2022 Workshop on Efficient Natural Language and Speech Processing*, 2022. arXiv:2107.02027.
- Richard E. Ladner and Michael J. Fischer. Parallel prefix computation. *Journal of the ACM*, 27(4):831–838, 1980.
- Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan, and Ion Stoica. RLlib: Abstractions for distributed reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2018. arXiv:1712.09381.
- Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. Safe and efficient off-policy reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. arXiv:1606.02647.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2203.02155.
- Fabio Pardo, Arash Tavakoli, Vitaly Levnik, and Petar Kormushev. Time limits in reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2018. arXiv:1712.00378.

- Aleksei Petrenko, Zhehui Huang, Tushar Kumar, Gaurav Sukhatme, and Vladlen Koltun. Sample factory: Egocentric 3D control from pixels at 100000 FPS with asynchronous reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020. arXiv:2006.11751.
- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016. arXiv:1511.05952.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. arXiv:1506.02438.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Jimmy T. H. Smith, Andrew Warrington, and Scott W. Linderman. Simplified state space layers for sequence modeling. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2208.04933.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. arXiv:2104.09864.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.
- Harm van Seijen, A. Rupam Mahmood, Pilarski Christos, Oussama Mourad, and Richard S. Sutton. True online temporal-difference learning. *Journal of Machine Learning Research*, 17(145):1–40, 2016.
- Ziyu Wang, Victor Bapst, Nicolas Heess, Volodymyr Mnih, Rémi Munos, Koray Kavukcuoglu, and Nando de Freitas. Sample efficient actor-critic with experience replay. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1611.01224.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. In *Communications of the ACM*, volume 52, pages 65–76, 2009.